Universidade Politécnica do Cávado e do Ave

Licenciatura em Engenharia de Sistemas Informáticos

Integração de Sistemas de Informação · 2026/2027

# Trabalho Prático I

## Integração e Análise de Dados de Videojogos

Processos ETL com Pentaho Data Integration (Kettle)

Aluno: Vitor Rezende· N.º 31521

Docente: Óscar Ribeiro

Repositório GitHub: https://github.com/Shuchuga85/Trabalho1ETL_31521

Vídeo de demonstração: https://vimeo.com/1238015643

10/2026

## Índice

1. Enquadramento e objetivo ..... 3
2. Fontes de dados e arquitetura ..... 3
3. Desenvolvimento dos processos ETL ..... 3
3.1. Importação e preparação das vendas ..... 4
3.2. Recolha paginada da RAWG ..... 5
3.3. Integração de videojogos e vendas ..... 5
3.4. Estatísticas e integração do XML ..... 6
4. Orquestração e execução ..... 7
5. Resultados e validação ..... 8
6. Dificuldades, limitações e trabalhos futuros ..... 9
7. Conclusão ..... 10
8. Referências e ligações ..... 10

## 1. Enquadramento e objetivo

Este trabalho foi desenvolvido no âmbito da unidade curricular de Integração de Sistemas de Informação e procura demonstrar a recolha, transformação e centralização de informação proveniente de fontes heterogéneas. O cenário escolhido é o dos videojogos: uma mesma obra pode surgir em plataformas diferentes e com títulos, datas e identificadores distintos, o que dificulta a comparação direta dos dados.

O objetivo é integrar metadados de videojogos com dados históricos de vendas, identificar correspondências e produzir estatísticas em ficheiros de saída. O processo foi implementado no Pentaho Data Integration (Kettle), usando transformações (.ktr) para tratar dados e um Job (.kjb) para coordenar a execução.

A solução não pretende prever vendas nem substituir serviços especializados. Trata-se de um protótipo didático de ETL, centrado na interoperabilidade dos formatos, na qualidade dos dados e na automação do processamento.

## 2. Fontes de dados e arquitetura

Foram selecionadas duas fontes principais e um ficheiro auxiliar local. A RAWG fornece metadados através de uma API REST em JSON; o conjunto vgsales.csv contém vendas históricas por título, plataforma e região; e um XML local fornece regras simples de correspondência entre géneros e categorias normalizadas.

| Origem | Formato | Papel no processo |
|---|---|---|
| RAWG Video Games API | JSON / REST | Identificação, data de lançamento e classificações dos jogos |
| vgsales.csv | CSV | Géneros, plataformas, editoras e vendas históricas por região |
| genre_mapping.xml | XML | Mapeamento de géneros para categorias uniformizadas |

Os dados tratados são guardados em SQL Server, na base de dados VideoGamesETL. As tabelas de staging mantêm os registos preparados de cada origem; a tabela Integrated_Games reúne os jogos RAWG e as vendas que puderam ser associadas.

## 3. Desenvolvimento dos processos ETL

O desenvolvimento foi organizado em quatro transformações independentes. A opção por processos separados simplifica os testes, permite identificar erros em cada fase e facilita a repetição das importações. O fluxo global segue a sequência indicada abaixo.

| Ordem | Ficheiro | Função |
|---|---|---|
| 1 | 01_import_sales.ktr | Ler, validar e persistir as vendas do CSV |
| 2 | 02_import_rawg_paginated.ktr | Consultar a API por páginas e persistir jogos RAWG |
| 3 | 03_integrate_games.ktr | Normalizar títulos e cruzar as duas origens |
| 4 | 04_generate_statistics.ktr | Agregações e exportação de relatórios, incluindo XML |

*Fluxo geral: CSV e API RAWG → tabelas de staging SQL → normalização e Merge Join → Integrated_Games → estatísticas CSV. O XML é utilizado no ramo de classificação de géneros.*

## 3.1. Importação e preparação das vendas

A transformação 01_import_sales.ktr lê o ficheiro vgsales.csv, seleciona os campos necessários e normaliza os respetivos tipos. O campo Year foi inicialmente recebido como texto, uma vez que a fonte podia conter o valor N/A; após a limpeza, passou a ser convertido para inteiro, admitindo NULL quando não existe ano válido.

Um filtro distingue registos válidos e inválidos. O fluxo principal calcula métricas derivadas a partir das vendas regionais, nomeadamente uma soma de controlo das vendas globais e percentagens regionais. Os registos são exportados para CSV e carregados na tabela dbo.Staging_Sales, preservando assim uma fonte estruturada para as fases seguintes.

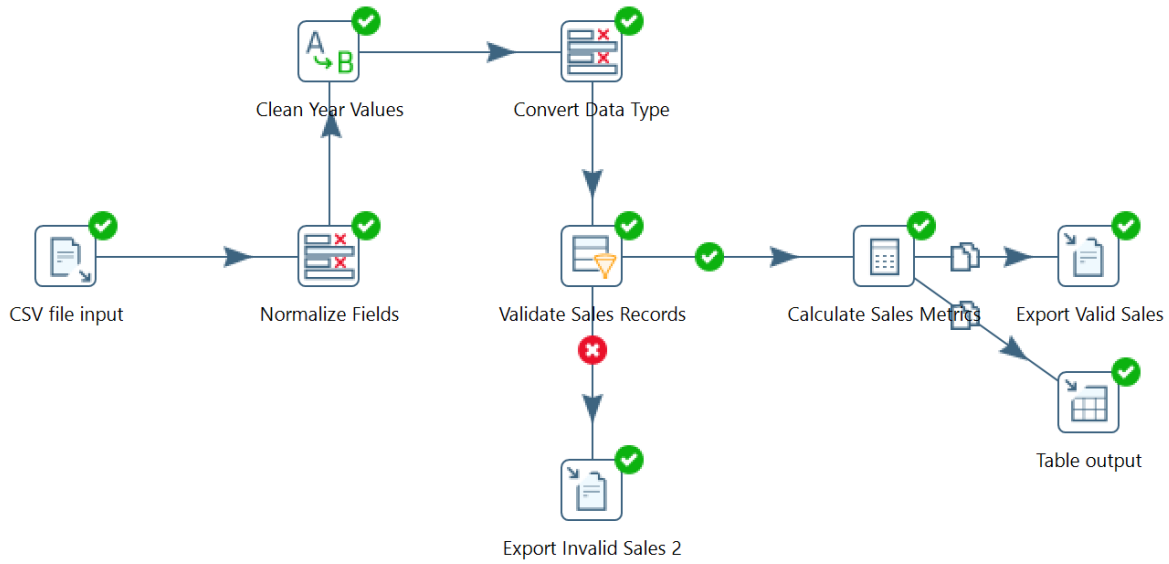


*Figura 1 — Transformação de importação e validação do CSV no Spoon.*

## 3.2. Recolha paginada da RAWG

A transformação 02_import_rawg_paginated.ktr utiliza Generate Rows e Add Sequence para construir números de página. Um REST Client efetua os pedidos HTTP e o JSON Input converte o array results em linhas com identificador, nome, data de lançamento, avaliação e outros metadados. Um filtro valida as respostas HTTP antes do processamento, evitando interpretar respostas de erro como dados de jogos.

Na execução de teste foram utilizadas cinco páginas com até 40 jogos cada, tendo sido obtidos 200 registos. Após seleção e conversão dos campos, estes são guardados em dbo.Staging_RawgGames. A chave de API é tratada como parâmetro e não deve constar do repositório.

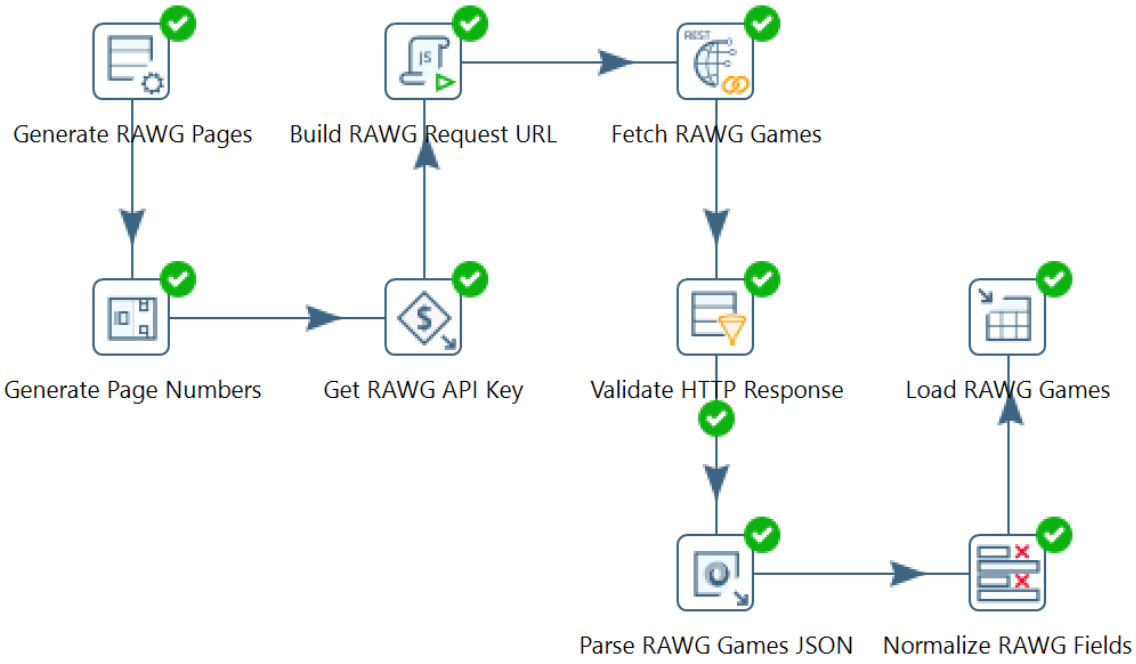



*Figura 2 — Transformação de recolha paginada da RAWG no Spoon.*

## 3.3. Integração de videojogos e vendas

A transformação 03_integrate_games.ktr lê as duas tabelas de staging e prepara chaves de correspondência baseadas no nome do jogo. São usadas operações de texto e a expressão regular [^a-z0-9] para retirar espaços e pontuação após a conversão para minúsculas. O ramo das vendas agrega os valores por título; ambos os fluxos são ordenados antes do Merge Join.

Foi utilizado um LEFT OUTER JOIN para preservar todos os jogos RAWG, mesmo quando não existe correspondência no dataset de vendas. Um campo lógico (has_sales_data) identifica os jogos enriquecidos. Numa execução confirmada, 87 dos 200 jogos encontraram correspondência, enquanto 113 ficaram sem associação.

Esta estratégia de correspondência é deliberadamente simples: títulos semelhantes, edições diferentes e plataformas distintas podem originar associações incompletas ou incorretas. Assim, os resultados devem ser interpretados como correspondências automáticas de uma primeira versão, e não como reconciliação perfeita de catálogos.

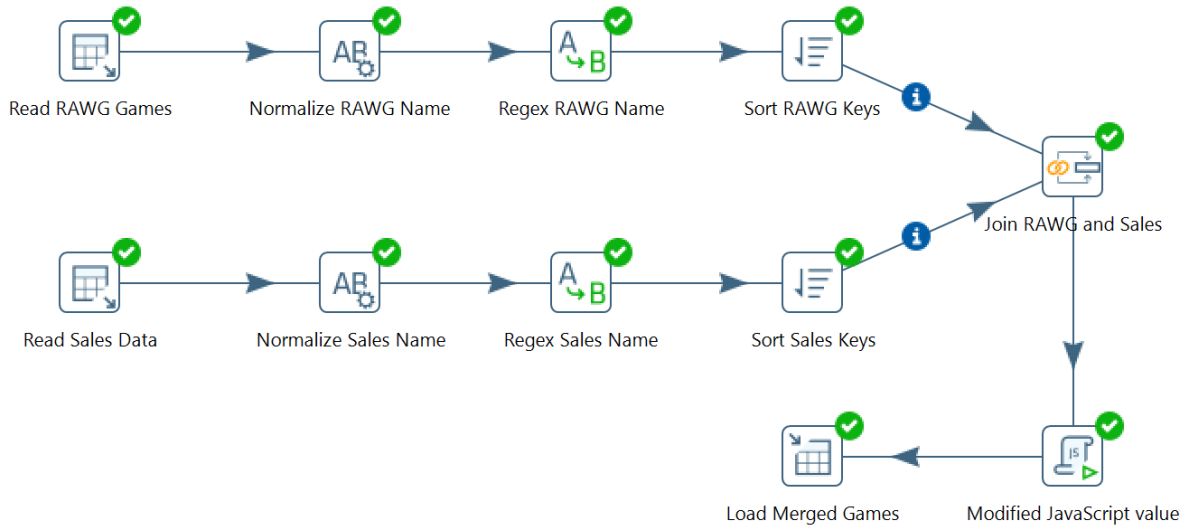



*Figura 3 — Normalização de nomes e Merge Join entre RAWG e vendas.*

## 3.4. Estatísticas e integração do XML

A transformação 04_generate_statistics.ktr produz relatórios a partir das tabelas SQL. Um fluxo agrupa as vendas por género, calcula totais e médias e ordena os resultados; outro resume o número de jogos com e sem dados de vendas, permitindo avaliar a taxa de associação obtida na integração.

Foi acrescentado um terceiro fluxo que lê genre_mapping.xml com o step Get data from XML. O Stream Lookup associa o campo Genre da tabela de vendas ao campo source do XML e devolve normalized_genre. Os registos são posteriormente agrupados por essa categoria e exportados para normalized_genre_statistics.csv.

Na configuração inicial, todas as categorias surgiam como Unmapped devido a um erro de sintaxe no XML, corrigido após validação da leitura. O ficheiro passou a participar efetivamente na transformação e não apenas a existir como exemplo do formato.

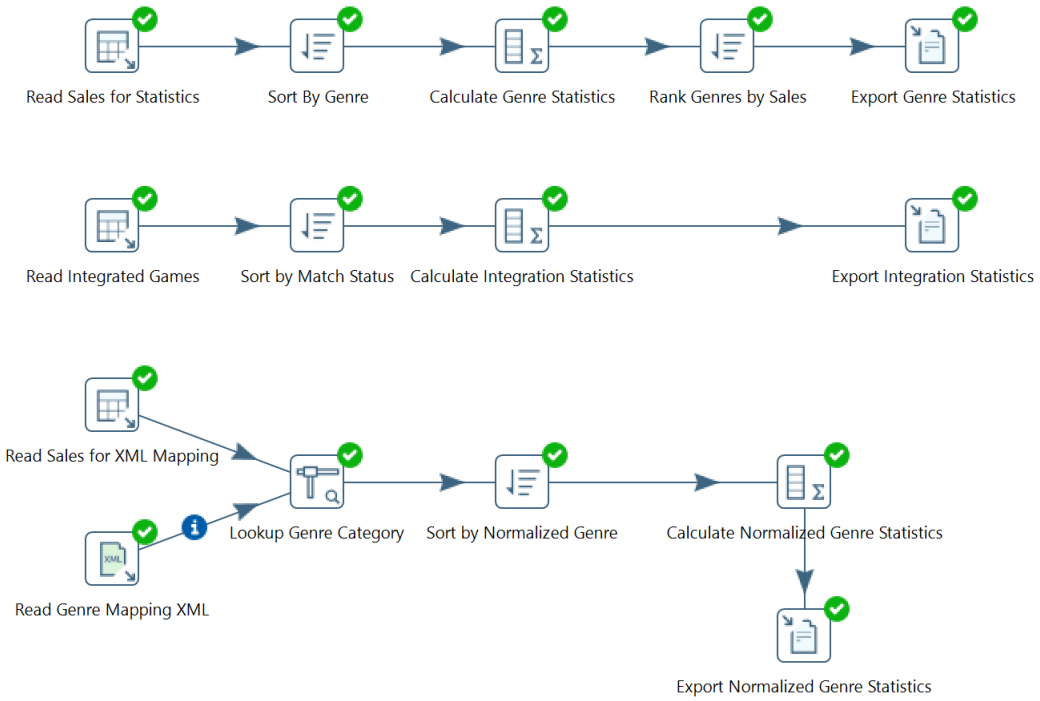



*Figura 4 — Fluxos de estatísticas, incluindo leitura XML e Stream Lookup.*

## 4. Orquestração e execução

O Job 00_main_job.kjb coordena as quatro transformações por ordem: importação do CSV, recolha da RAWG, integração e geração dos relatórios. Os caminhos de sucesso permitem avançar para a etapa seguinte; os caminhos de falha conduzem a uma entrada de aborto. Deste modo, evita-se continuar a execução caso uma fase essencial termine com erro.

A chave RAWG_API_KEY é fornecida como parâmetro de execução. As transformações utilizam caminhos relativos sempre que possível para facilitar a transferência do projeto. A validação final deve ser feita a partir do Job, verificando o log completo e não apenas as execuções isoladas das transformações.

Para assegurar a rastreabilidade do processamento, foi configurada a exportação de logs das transformações para ficheiros na pasta logs/. Estes registos permitem consultar a execução de cada etapa, incluindo mensagens de processamento, eventuais erros e informação temporal, facilitando a identificação de problemas sem depender exclusivamente da consola do Spoon.

No final da quarta transformação, o Job executa uma entrada Mail que envia automaticamente uma notificação de sucesso por correio eletrónico, confirmando a conclusão do processo. Caso ocorra uma falha numa das transformações, mantém-se o encaminhamento para Abort job; nesta versão não foi implementado o envio de e-mails de erro.

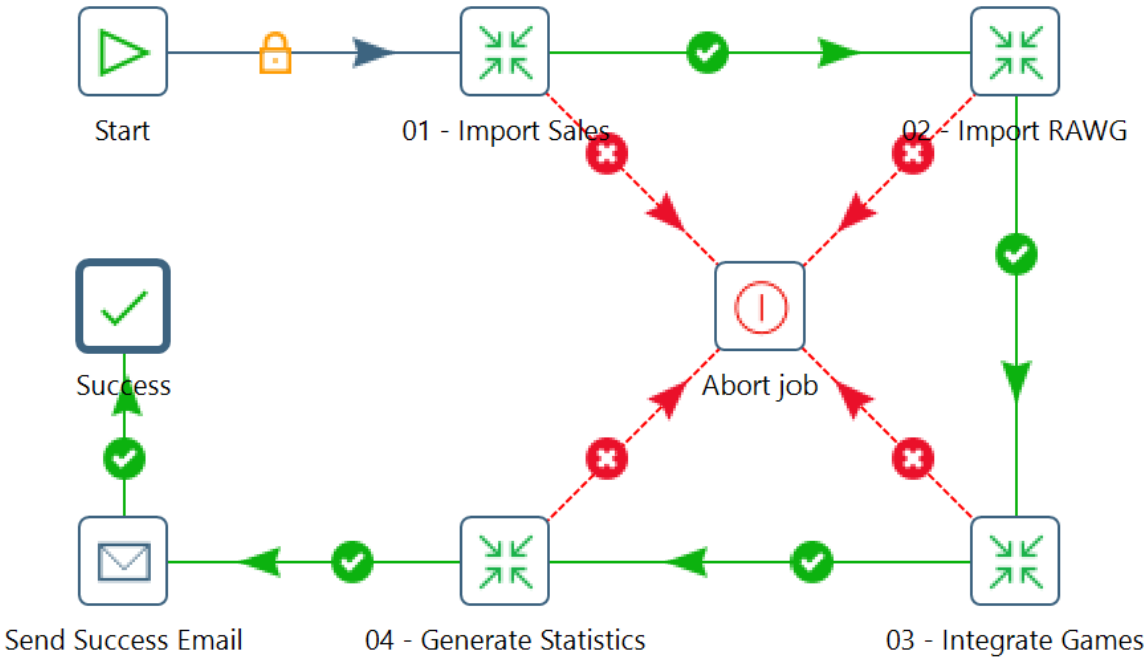



Figura 5 — Job principal, caminhos de sucesso/erro e notificação por e-mail.

## 5. Resultados e validação

Os testes realizados confirmaram a importação de dados CSV e JSON para SQL Server, o cruzamento entre fontes, a utilização de XML para normalização de géneros e a exportação automática de estatísticas. Na experiência de integração registaram-se 200 jogos RAWG, 87 associados às vendas históricas e 113 sem correspondência.

| Indicador | Resultado |
|---|---|
| Jogos recolhidos da RAWG (teste) | 200 |
| Jogos com dados de vendas associados | 87 |
| Jogos sem correspondência | 113 |
| Taxa de correspondência | 43,5% |
| Géneros normalizados pelo XML | 87% |
| Execução final do Job | Concluído com sucesso — ver Figura 6 |

Nos ficheiros exportados, o indicador has_sales_data pode surgir como Y/N porque é interpretado como Boolean pelo Pentaho. Valores de vendas não disponíveis podem surgir em branco, correspondendo a NULL na origem. Estes comportamentos não representam, por si só, falhas do processamento.

Foram utilizados previews dos steps, contagens no SSMS e comparação dos totais dos relatórios com consultas SQL. Antes da entrega, deverá ser repetida a execução completa do Job, confirmando que termina sem erros, que as tabelas têm os totais esperados e que os ficheiros CSV foram atualizados.

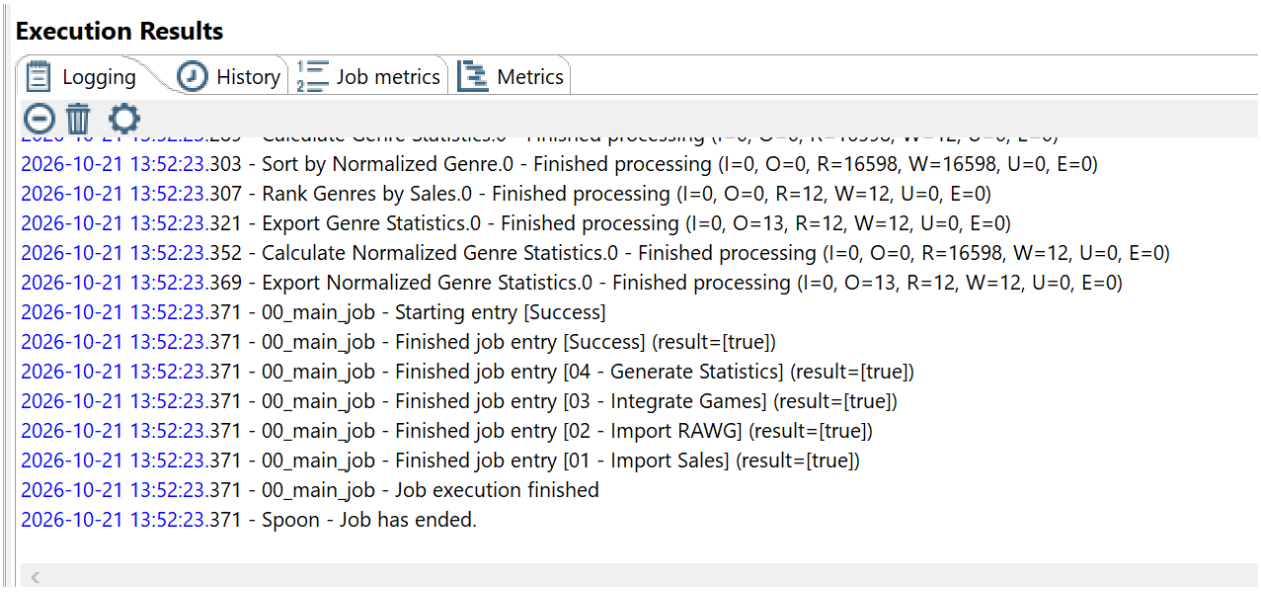



*Figura 6 — Captura do log final do Job concluído com sucesso.*

## 6. Dificuldades, limitações e trabalhos futuros

Durante o desenvolvimento, foram encontrados problemas reais de tratamento de dados: valores N/A na coluna Year, um caminho JSONPath incorreto para released, ausência de ordenação nas entradas do Merge Join e um erro de sintaxe no ficheiro XML. A resolução passou pela limpeza prévia à conversão, correção dos caminhos, ordenação explícita dos fluxos e validação isolada da leitura XML.

A principal limitação funcional é o cruzamento pelo nome normalizado do jogo, que não considera adequadamente plataformas, edições e variações de títulos. Além disso, a amostra RAWG foi limitada a 200 jogos para os testes, e o dataset de vendas é histórico. Os valores agregados não devem ser interpretados como vendas atuais nem como um catálogo completo.

Como evolução, seria útil aumentar a cobertura da recolha API, introduzir correspondências adicionais por plataforma/ano, implementar verificações mais rigorosas de duplicados, melhorar os registos de auditoria e criar um dashboard. Estas melhorias não são indispensáveis para o objetivo didático da versão atual.

## 7. Conclusão

A solução desenvolvida demonstra um processo ETL completo de integração de dados de videojogos. O Pentaho foi utilizado para consumir fontes em CSV, JSON e XML, aplicar limpeza e normalização, realizar joins e lookups, calcular agregações, persistir registos em SQL Server e exportar relatórios. A execução coordenada por um Job permite repetir o processo de forma organizada e observar erros por etapa.

A experiência mostrou que integrar sistemas não consiste apenas em copiar ficheiros: é necessário compatibilizar formatos, tratar informação incompleta, definir chaves de correspondência e validar resultados. Apesar das limitações da primeira versão, os objetivos principais do trabalho foram atingidos no cenário experimental descrito.

## 8. Referências e ligações

Pentaho Data Integration / Kettle — https://github.com/pentaho/pentaho-kettle

RAWG API — https://rawg.io/apidocs

Dataset Video Game Sale — https://zenodo.org/records/5898311/files/vgsales.csv?download=1

Microsoft SQL Server — https://learn.microsoft.com/sql/

Repositório do trabalho — https://github.com/Shuchuga85/Trabalho1ETL_31521

Vídeo de demonstração / QR Code — https://vimeo.com/1238015643